

TECHNICAL SPECIFICATION

Data Alchemist

Solution Architecture & Technical Specification — Agentic Data Quality
& Trust Platform

Document type: Solution Architecture & Technical Specification

Prepared for: SCALEX Pursuit 2026 — Supporting Artefact

Scope: Runtime architecture, connector abstraction, agent & AI layer, data model

Verification: Every count and claim in this document is drawn directly from the source repository at time of writing

Table of Contents

1. Executive Summary	3
2. System Architecture Diagram	4
3. Layer-by-Layer Component Description	5
4. Data Model — Entity-Relationship Overview	6
5. Technology Stack	7
6. Security & Compliance Posture	7

1. Executive Summary

Data Alchemist is an agentic data quality and trust platform built on a five-layer architecture: a browser-transpiled React SPA, a FastAPI application layer, an intelligence layer combining one LangGraph multi-step agent with three governed LLM-service modules, a connector abstraction layer supporting five database platforms, and a PostgreSQL metadata store spanning 36 versioned schema migrations.

The platform's central design decision is **strict separation between AI-assisted authoring and deterministic execution**: profiling, rule recommendation, and anomaly/failure explanation use an LLM (via a swappable LiteLLM gateway); rule *execution* and scoring are pure, deterministic SQL with no LLM involvement. Every AI output is either structurally validated before a human sees it, or requires explicit human approval before it changes system state.

Verified footprint at time of writing: 11 frontend screens · 12 API routers · 94 REST endpoints · 5 live database connectors · 4 backend agent/service modules · 36 PostgreSQL schema migrations · 30 database tables.

This document describes the architecture as implemented, not as planned. Every figure above is a direct count from the source repository (module inventory in Section 3, endpoint-level detail in the accompanying Endpoint & Module Inventory workbook).

Agentic Data Quality & Trust Platform — five-layer architecture, verified against source



Layer-by-Layer Component Description

01 · Client Layer — React 18 SPA

11 workflow screens (Profiling, Rules, Execution, Anomalies, Dashboard, Lineage/Impact, Simulator, Intel, Metadata, Connections, Tasks), transpiled in-browser via Babel — there is no frontend build step. Served through an nginx reverse proxy configured with `proxy_buffering off` to support real-time Server-Sent Events for long-running agent operations (profiling runs, simulation streams).

02 · Application Layer — FastAPI

12 API routers exposing 94 REST endpoints. Authentication supports two paths simultaneously: local email/password (bcrypt-hashed, JWT bearer tokens) and Microsoft Entra ID SSO (OAuth2 Authorization Code + PKCE, no client secret in the browser). Every table in the metadata store — with one deliberate exception (the shared simulation-scenario library) — carries a `connection_id` foreign key, so every route is inherently tenant-scoped.

03 · Intelligence Layer — Agents & Services

Profiling Agent

The only true LangGraph StateGraph in the codebase: an 11-node pipeline (`fetch_schema` → `compute_null_stats` → `compute_distinct` → `check_referential_integrity` → `check_key_duplicates` → `compute_formats` → `compute_numerics` → `detect_duplicates` → `score_table` → `identify_risks` → `generate_summary`), streaming progress as Server-Sent Events.

Rule Recommendation

LLM-proposed rules in structured JSON, validated by 7 deterministic static-analysis checks before reaching a human reviewer — inverted null checks, uncorrelated EXISTS, malformed boolean combinations, unverified table/column references, and more.

Execution Engine

Deliberately contains no LLM call. Runs already-approved rules as parameterized SQL and computes a weighted quality score — keeping the auditable execution path fully deterministic.

Explainability

Converts technical anomaly and rule-failure records into plain-English narratives via LLM, with every call logged (model, tokens, cost) to the AI usage ledger.

LiteLLM Gateway: every LLM call in the platform routes through one abstraction layer. The active model (Claude Sonnet by default) is swapped for GPT-4o, Gemini, or a local Ollama model via a single environment variable — zero code changes at any call site.

04 · Connector Abstraction Layer

A single BaseConnector interface (`test` · `list_schemas` · `list_tables` · `describe_table` · `query`) with five concrete implementations: SQL Server, Snowflake, Databricks, PostgreSQL, DuckDB. Each connector owns its own identifier-quoting rule (brackets, backticks, or ANSI double-quotes) and FROM-clause construction, so every stats query issued by the profiling/rules/execution layers is dialect-correct without special-casing upstream.

05 · Data Source Layer

Client warehouses are accessed read-only through the connector layer; credentials are Fernet-encrypted before they ever reach the metadata store, and are never written to logs or returned by any API response.

Technology Stack

Layer	Technology	Notes
Frontend	React 18 + Babel (in-browser)	No bundler/build step; edit JSX, reload browser
Reverse proxy	nginx	proxy_buffering off for SSE streaming
Backend framework	Python 3.11 · FastAPI	12 routers, 94 endpoints, OpenAPI/Swagger auto-generated
Agent orchestration	LangGraph	StateGraph used for the Profiling Agent's 11-node pipeline
LLM abstraction	LiteLLM	Anthropic, OpenAI, Azure OpenAI, Gemini, Ollama — provider swap via env var
Metadata database	PostgreSQL	36 versioned migrations, auto-applied at startup
Auth	bcrypt + JWT · Microsoft Entra ID (OAuth2/PKCE)	Both paths supported simultaneously
Credential storage	Fernet symmetric encryption	Connection credentials encrypted at rest, decrypted only in-process
Containerization	Docker Compose	postgres · backend · frontend services
Connectors	SQL Server, Snowflake, Databricks, PostgreSQL, DuckDB	Common BaseConnector interface

6. Security & Compliance Posture

- **Credentials:** Fernet-encrypted at rest in the metadata database; never returned by any API response or written to logs.
- **Authentication:** bcrypt password hashing, JWT bearer tokens, and Microsoft Entra ID SSO (OAuth2 Authorization Code + PKCE — no client secret exposed to the browser).
- **Tenant isolation:** 28 of 29 satellite tables carry a connection_id foreign key; every API route operates within that scope.
- **Auditability:** an immutable audit_trail table records every human decision (approve, edit, reject, suppress, escalate, promote, demote, snooze, acknowledge) with old/new value snapshots.
- **AI governance:** every LLM call (model, token count, cost) is logged to ai_usage_log; LLM-proposed rules pass 7 static-analysis checks before any human review.
- **Data minimization:** connectors read schema metadata and bounded samples for profiling; there is no bulk-export path off the platform.